

Idempotency & Exactly-Once Delivery

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

Chapter 8 ended with a promise: at-least-once delivery, the practical default for most message queues, can be made to *behave* like exactly-once from the application's point of view, even though the underlying infrastructure can't strictly guarantee it. The technique that makes this possible is **idempotency**, and it's one of the highest-leverage concepts in this entire program — it shows up constantly, not just in message queues but in any situation involving retries, which is to say, almost everywhere in a distributed system.

An operation is **idempotent** if performing it multiple times produces the exact same result and side effects as performing it exactly once. HTTP itself already encodes this idea: GET, PUT, and DELETE are specified to be idempotent (fetching, fully replacing, or deleting the same resource repeatedly should leave the system in the same end state as doing it once), while POST is explicitly *not* idempotent by default, because "create a new thing" run twice naturally creates two things. This is precisely why "charge this card \$500" as a POST is dangerous to retry blindly — a network timeout that hides a successful charge from the client, followed by an automatic retry, can genuinely charge the customer twice.

Backend engineer analogy

Think about a Laravel migration: running **php artisan migrate** twice doesn't create the tables twice — Laravel tracks which migrations have already run and skips them. That's idempotency by design. Compare that to running a raw **INSERT INTO users ...** SQL script twice, which happily creates two duplicate rows. The difference between those two experiences is exactly the difference idempotency keys are meant to engineer into your own API endpoints deliberately.

The idempotency key pattern

The standard technique for making a naturally non-idempotent operation (like "create a payment") safe to retry is the **idempotency key**: the client generates a unique identifier (a UUID, typically) for a given logical operation and sends it along with the request, usually as a header (e.g., **Idempotency-Key: 8f14e...**). The server, on receiving a request, first checks whether it has already seen and processed that exact key. If it has, the server returns the *original* result immediately without repeating the underlying side effects (charging the card again). If it hasn't, the server processes the request normally and stores the key alongside the result, ready to short-circuit any future retry with the same key. Critically, the key must represent a stable client-side retry, not be regenerated on every retry attempt — if the client generates a new key each time it retries, the server has no way to recognize the retry as a duplicate.

Implementing this correctly requires care around concurrency: what happens if two requests carrying the same idempotency key arrive nearly simultaneously (a client that fires a retry before the first attempt has even finished)? A robust implementation uses a database-level unique constraint or a

distributed lock on the idempotency key, so the second concurrent request either waits for the first to finish and then returns its result, or is rejected outright, rather than both requests racing to perform the side effect independently.

It's worth being explicit about the relationship between this chapter's title's two halves. **Exactly-once delivery** is a property you'd want the messaging infrastructure itself to guarantee — and as Chapter 8 noted, this is genuinely hard to achieve end-to-end, especially once a consumer's side effect reaches outside the messaging system's own boundary (like calling a third-party payment API). **Idempotency** is a property you engineer into your own operations so that even imperfect, at-least-once delivery produces exactly-once *effects*. In practice, virtually every production system takes the idempotency route rather than betting on infrastructure-guaranteed exactly-once delivery, because idempotency is achievable with well-understood, application-level techniques, while true end-to-end exactly-once delivery generally isn't, once external systems are involved.

This pattern generalizes well beyond payments. Webhook handlers (a third-party service calling your API to notify you of an event) should be idempotent, because webhook senders commonly retry on any non-2xx response or timeout, meaning your endpoint will eventually receive the same webhook more than once by design, not by accident. Background job consumers reading from an at-least-once queue need idempotent processing for the exact reason Chapter 8 described. Even client-side "double-click the submit button" protection is a smaller-scale version of the same underlying problem: the same logical action arriving at the server more than once.

2. Real-World Example / Case Study

Razorpay's and Stripe-style payment gateway APIs explicitly document and require idempotency keys on payment-creation endpoints precisely because network failures between a merchant's server and the payment gateway are common and expected — a merchant's checkout code retrying a timed-out payment request with the same idempotency key is guaranteed to charge the customer exactly once, no matter how many times the retry fires.

PhonePe's and Tabby's webhook-based order-status and payment-confirmation notifications to merchant systems are, by the nature of webhooks, delivered at-least-once — a merchant's webhook handler that isn't idempotent risks marking an order as paid, then paid again, then paid a third time, each time triggering downstream actions (like sending a duplicate confirmation email) that shouldn't repeat.

Swiggy's and Talabat's order-placement flow on a flaky mobile network is a very tangible, everyday example: a customer taps "place order," the request times out on their end (perhaps due to a weak connection) before the confirmation reaches their phone, and the app retries automatically. Without an idempotency key tied to that specific order attempt, this common, ordinary network hiccup would place two identical orders and charge the customer twice — a bug category serious enough that most food delivery and e-commerce checkout flows treat idempotent order creation as a hard requirement, not a nice-to-have.

3. Diagram

Without idempotency key:	With idempotency key:
<pre> Client -- POST /charge --> (times out, client doesn't know if it succeeded) Client retries -- POST /charge --> Server charges card AGAIN RESULT: customer charged TWICE </pre>	<pre> Client -- POST /charge Idempotency-Key: abc123 --> (times out) -- Client retries SAME request with SAME key: abc123 --> Server sees key "abc123" already processed -> returns ORIGINAL result, does NOT charge again RESULT: charged ONCE </pre>

4. Trade-offs Table

Approach	Guarantee	Implementation cost
No idempotency handling	None — retries can duplicate side effects	Lowest, but riskiest for financial/critical ops
Idempotency key + DB unique constraint	Duplicate requests return original result, no re-execution	Moderate: schema change + lookup logic
Idempotency key + distributed lock	Also handles concurrent duplicate requests safely	Higher: needs a lock/coordination service
Rely on infra-level exactly-once (e.g. Kafka transactions)	Strong within the messaging system's own boundary only	Doesn't cover external side effects (e.g. calling a payment API)

5. Common Interview Questions

Q1: "How would you make a 'create payment' API safe to retry?"

Model approach: Require an idempotency key from the client, store it with the operation's result, and on any retry with the same key return the stored result instead of re-executing the charge; mention the unique-constraint or locking mechanism needed to handle near-simultaneous duplicate requests correctly.

Q2: "Is GET idempotent? Is POST? Why does this distinction matter for retries?"

Model approach: GET, PUT, DELETE are specified idempotent by the HTTP standard (repeating them leaves the same end state); POST is not, by default, because it typically creates a new resource each time. This matters because HTTP clients, proxies, and load balancers are generally safe to auto-retry idempotent methods, but must not blindly auto-retry POST without additional protection like an idempotency key.

Q3: "Your webhook handler processed the same event twice and sent two emails. How do you fix it?"

Model approach: Diagnose this as expected at-least-once webhook delivery behavior, not a bug in the sender, and fix it by tracking processed event IDs (most webhook payloads include one) in your own database, skipping any event ID you've already processed before triggering side effects like sending email.

Q4: "Why is true exactly-once delivery so hard to achieve across a distributed system?"

Model approach: Explain that guaranteeing exactly-once requires coordinating the message delivery AND the consumer's side effect as a single atomic unit, which is straightforward within one system's boundary (like Kafka's internal transactions) but breaks down the moment the side effect reaches an external system (a third-party API call) that isn't part of that same transactional boundary — which is exactly why idempotency at the application layer is the practical, proven alternative.

6. Practice Exercises

- 1 Design the database schema and request flow for an idempotency-key-protected 'place order' endpoint, including how you'd handle two near-simultaneous requests with the same key.
- 2 List three HTTP endpoints in a typical e-commerce API and classify each as naturally idempotent or not, explaining your reasoning for each.
- 3 A teammate says 'we use Kafka, so we don't need to worry about duplicate processing.' Write a short rebuttal explaining specifically where their reasoning breaks down.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Episode 12: Idempotency Basics System Design Fundamentals English"	Clear foundational explanation of idempotency in system design.
English	"Idempotency in APIs Explained Why It Matters + Code Example"	Shows a concrete code-level implementation of the idempotency key pattern.
Hindi	"Idempotent APIs - Why Idempotency Is Critical in Modern APIs Explained in Hindi Day 77/100"	Hindi explanation focused specifically on API idempotency.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers idempotency and retries as part of a full bilingual system design course.

8. Key Takeaways

- Idempotent operations produce the same result and side effects no matter how many times they're repeated.
- HTTP's GET/PUT/DELETE are specified idempotent; POST is not, which is exactly why naive POST retries are risky.
- The idempotency key pattern makes non-idempotent operations (like payments) safe to retry.
- Concurrent duplicate requests need a unique constraint or lock, not just a simple lookup, to be handled correctly.
- Webhook handlers and queue consumers must be idempotent, since at-least-once delivery is the practical norm.
- True infra-level exactly-once delivery rarely extends to external side effects like third-party API calls.
- Idempotency is how real systems make imperfect, at-least-once delivery behave like exactly-once in practice.